Studio Nx DB 스크립트 가이드

항목	내용
DBMS	MySQL 8.0 (CHECK 제약 사용 → 8.0.16 이상 필요)
Database	`studionx` (문자셋 `utf8mb4` / `utf8mb4_0900_ai_ci`)
테이블 수	34 개
설계 원본	`docs/DB.md` (컬럼 상세·설계 이유), `docs/ERD.md` (관계 설명)
작성일	2026-10-01 (최종 수정 2026-10-02)
상태	로컬 DB 에 01~06 실행 완료, 2026-10-02 변경 SQL(§2.5)까지 반영 → 34 개 테이블 확인

1. 파일 구성

순서	파일	내용	테이블 수	실행 시기 (구현 계획 기준)
0	`00_create_database.sql`	database `studionx` + 앱 계정 `studionx_app` 생성	—	개발 시작 시 (root 계정)
1	`01_phase1_account.sql`	회원, 문의, Development 공지, 프로젝트, 프로젝트 멤버	5	1 단계 S1-02
2	`02_phase2_sequencer.sql`	샘플·태그·플러그인·트랙·디바이스·패턴·클립·노트·오토메이션·템플릿·UI 설정	16	1 단계 S1-05 ~
3	`03_phase3_export.sql`	오디오 출력 작업	1	1 단계 S1-11
4	`04_phase4_separation.sql`	악기 분리 작업·결과	2	2 단계 S2-14
5	`05_phase5_ai.sql`	레퍼런스·AI 분석·추천·Nyxi 대화	5	2 단계 S2-12
6	`06_phase6_collaboration.sql`	초대·편집 이력·톡방·코멘트·스냅샷	5	3 단계 S3-01
99	`99_drop_all_tables.sql`	모든 테이블 삭제 (개발용 초기화)	—	필요할 때만

- 파일 번호 순서대로 실행해야 합니다. 뒤 파일의 테이블이 앞 파일의 테이블을 외래키로 참조합니다.

- 한 번에 모두 만들어도 되고, 구현 단계에 맞춰 필요한 파일만 실행해도 됩니다.
- 확인한 것: 34 개 테이블 중복 없음, 모든 외래키 대상이 먼저 생성됨, MySQL 문법 파싱 통과, 로컬 MySQL 8.0 에서 실제 실행.

2. 실행 방법

> **PowerShell 에서는 `<`를 쓸 수 없습니다** (`The '<' operator is reserved for future use.` 오류).

> 아래 명령은 MySQL 의 `source` 명령을 쓰는 방식이라 PowerShell, 명령 프롬프트(cmd), Git Bash 어디서나 됩니다. 비밀번호는 실행 후 입력합니다.

> 경로는 **`studionx/` 폴더에서 실행하는 기준**입니다. 저장소 맨 위(`StudioNx/`)에서 실행하면 `db/` 앞에 `studionx/`를 붙입니다.

2.1 database 와 계정 만들기 (최초 1 회)

1. `00_create_database.sql` 의 `<APP_PASSWORD>`를 실제 비밀번호로 바꿉니다.
2. 같은 비밀번호를 `studionx/.env` 의 `DB_PASSWORD` 에 넣습니다.
3. root 계정으로 실행합니다.

```
mysql -u root -p -e "source db/00_create_database.sql"
```

2.2 테이블 만들기

`studionx/` 폴더에서 앱 계정으로 실행합니다.

```
mysql -u studionx_app -p studionx -e "source db/01_phase1_account.sql"
mysql -u studionx_app -p studionx -e "source db/02_phase2_sequencer.sql"
mysql -u studionx_app -p studionx -e "source db/03_phase3_export.sql"
mysql -u studionx_app -p studionx -e "source db/04_phase4_separation.sql"
mysql -u studionx_app -p studionx -e "source db/05_phase5_ai.sql"
mysql -u studionx_app -p studionx -e "source db/06_phase6_collaboration.sql"
```

2.3 확인

```
-- 테이블 수 (전부 실행했다면 34)
SELECT COUNT(*) FROM information_schema.tables WHERE table_schema = 'studionx';


-- 외래키 목록
SELECT table_name, constraint_name, referenced_table_name
FROM information_schema.referential_constraints
WHERE constraint_schema = 'studionx' ORDER BY table_name;
```

서버 실행 후 `http://localhost:8000/api/v1/health/db` 가 `{"db":"ok"}`이면 백엔드 연결도 정상입니다.

2.4 초기화 (개발용)

```
mysql -u studionx_app -p studionx -e "source db/99_drop_all_tables.sql"
```

모든 데이터가 지워집니다. 운영 DB 에서는 절대 실행하지 마세요. 이후 01~06 을 다시 실행하면 빈 스키마가 됩니다.

2.5 변경 SQL 실행 (이미 테이블이 있는 DB)

01~06 을 예전에 실행한 DB 는 테이블을 다시 만들지 않고 `db/migrations/`의 변경 SQL 만 실행합니다. 목록은 §6.1 에 있습니다.

```
mysql -u studionx_app -p studionx -e "source db/migrations/2026-10-02_contact_devnotes.sql"
```

확인: §2.3 의 테이블 수가 34 이고, 아래 명령 결과로 한 줄이 나오면 정상입니다.

```
mysql -u studionx_app -p studionx -e "SHOW COLUMNS FROM inquiries LIKE 'privacy_agreed_at';"
```

3. 테이블 목록

3.1 `01_phase1_account.sql`

테이블	설명	주요 키
`users`	회원, 저장 용량	PK user_id · UQ email, nickname
`inquiries`	홈 화면 문의 (이름 선택, 개인정보 동의 시각 필수)	FK user_id (NULL = 비회원)
`dev_notes`	메인 페이지 Development 공지 (관리자만 작성)	FK author_id
`projects`	시퀀서 프로젝트	FK owner_id, template_id
`project_members`	참여자·권한 (생성 시 OWNER 등록)	PK (project_id, user_id)

3.2 `02_phase2_sequencer.sql`

테이블	설명	주요 키
`samples`	샘플 라이브러리 (업로드·녹음·출력·분리 결과)	FK owner_id
`tags`	태그 (장르·악기·분위기)	UQ (category, name)
`sample_tags`	샘플-태그 연결	PK (sample_id, tag_id)
`plugins`	웹 가상악기·이펙트 목록, 카드 아트 키	UQ code

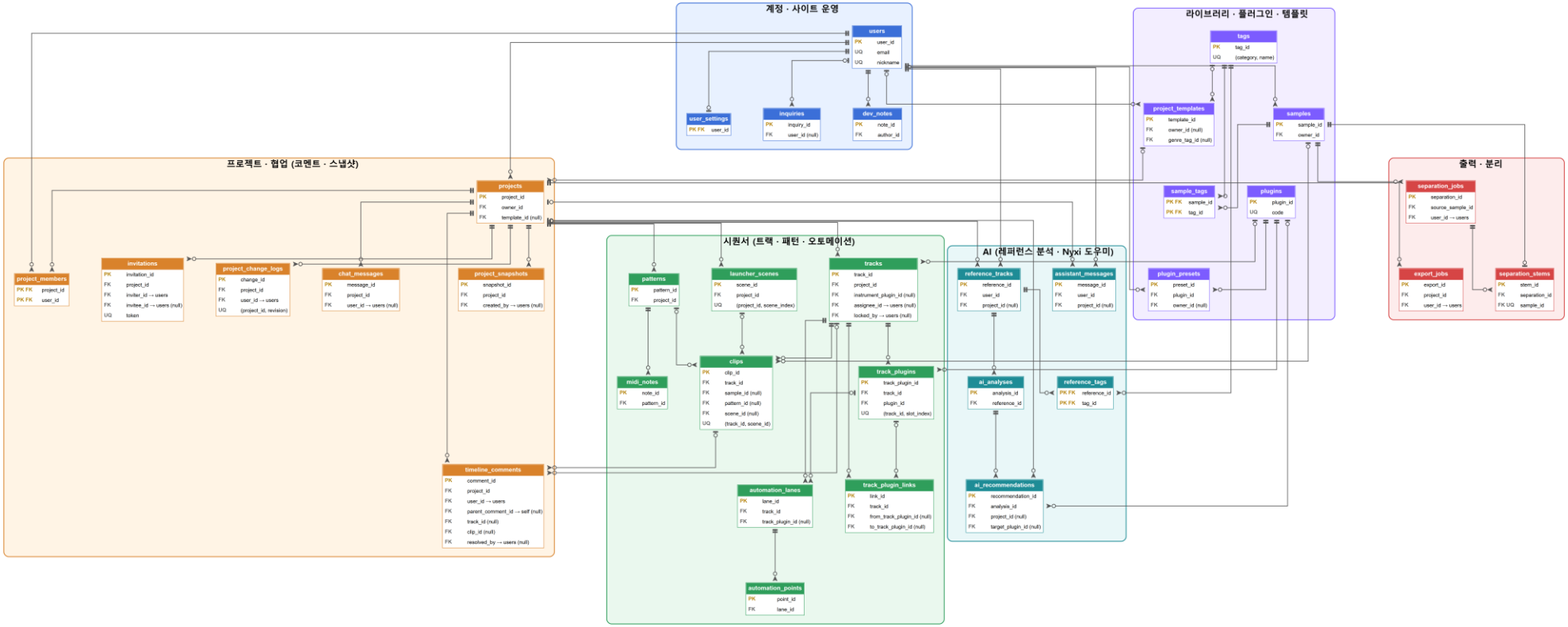
테이블	설명	주요 키
plugin_presets	플러그인 프리셋	FK plugin_id, owner_id
tracks	트랙 (AUDIO / MIDI / MASTER), 담당자·잠금	FK project_id, instrument_plugin_id
track_plugins	트랙 디바이스 (체인 순서, 패처 노드 위치)	UQ (track_id, slot_index)
track_plugin_links	패처 연결선	FK track_id, from/to_track_plugin_id
patterns	MIDI 패턴 (링크 클립이 공유)	FK project_id
launcher_scenes	클립 런처 씬	UQ (project_id, scene_index)
clips	타임라인·런처 클립	FK track_id, sample_id, pattern_id, scene_id
midi_notes	패턴의 노트	FK pattern_id
automation_lanes	오토메이션 레인	FK track_id, track_plugin_id
automation_points	오토메이션 포인트	FK lane_id
project_templates	프로젝트 템플릿	FK owner_id, genre_tag_id
user_settings	테마·단축키·캐릭터·팁·튜토리얼 설정	PK = FK user_id

3.3 03 ~ 06

파일	테이블	설명
03	export_jobs	오디오 출력 작업 (형식, 진행률, 결과 파일)
04	separation_jobs	악기 분리 작업
04	separation_stems	분리 결과 stem (결과 파일은 samples 에 등록)
05	reference_tracks	레퍼런스 트랙 (링크 또는 파일)
05	reference_tags	레퍼런스-장르 태그 연결
05	ai_analyses	AI 분석 결과 (BPM, 키, LUFS 등)
05	ai_recommendations	AI 추천 설정 (프리셋·믹싱·마스터링)
05	assistant_messages	Nyxi 도우미 대화 기록

파일	테이블	설명
06	invitations	프로젝트 초대
06	project_change_logs	실시간 편집 변경 이력
06	chat_messages	프로젝트 톡방
06	timeline_comments	타임라인 위치 코멘트·답글
06	project_snapshots	버전 스냅샷

4. ERD



박스에는 PK·FK·UQ 컬럼만 표시됩니다. 전체 컬럼은 `docs/DB.md` §3 을 보세요.

5. 꼭 알아둘 규칙

1. **시간 단위는 tick** (PPQ = 960, 4 분음표 1 개 = 960). `midi_notes.start_tick` 은 패턴 시작 기준입니다.
2. **MIDI 노트는 패턴에 저장**되고, MIDI 클립은 `pattern_id` 로 패턴을 가리킵니다. 같은 패턴을 가리키는 클립끼리 링크 클립입니다.
3. **마스터도 트랙**입니다 (`track_type = 'MASTER'`, 프로젝트당 1 개, 프로젝트 생성 시 함께 INSERT).
4. **프로젝트 생성 시 함께 넣을 행**: `project_members` (OWNER), `tracks` (MASTER).
5. **소프트 삭제**: `users`, `projects`, `samples`, `chat_messages`, `timeline_comments`, `dev_notes` 는 `deleted_at` 으로 표시만 합니다.
6. **오디오 파일은 DB 에 넣지 않습니다**. `storage/` 기준 상대 경로만 저장합니다.
7. `02` 파일 안의 `ALTER TABLE projects` 는 `project_templates` 가 생긴 뒤에 `projects.template_id` 외래키를 추가하는 문장입니다.

5.1 CHECK 제약

테이블	제약	의미
`clips`	`chk_clips_kind`	MIDI 클립 = pattern_id 필수·sample_id 없음 / AUDIO 클립 = 그 반대
`midi_notes`	`chk_notes_pitch`, `chk_notes_velocity`	음높이 0~127, 벨로시티 1~127
`automation_points`	`chk_points_curve`	곡선 정도 -1.00 ~ 1.00
`reference_tracks`	`chk_ref_source`	LINK 면 source_url, FILE 이면 file_path 필수

5.2 삭제 정책 요약

경우	동작
프로젝트 삭제	트랙·클립·패턴·노트·디바이스·오토메이션·씬·멤버·초대·이력·채팅·코멘트·스냅샷·출력이 함께 삭제 (CASCADE)
프로젝트 삭제 시 레퍼런스·AI 추천·Nyxi 대화	`project_id` 만 NULL (SET NULL)

경우	동작
사용 중인 샘플·패턴	바로 삭제 불가 (RESTRICT) — 샘플은 소프트 삭제
트랙·클립 삭제 시 코멘트	코멘트는 남고 `track_id`/`clip_id` 만 NULL

6. 변경할 때의 순서

1. `docs/DB.md` 를 먼저 고칩니다 (설계 원본).
2. 같은 내용을 `db/0X_*.sql` 에 반영합니다.
3. 이 문서(md·docx)와 `docs/ERD.png` 를 갱신합니다.
4. 이미 데이터가 있는 DB 라면 테이블을 다시 만들지 말고 `ALTER TABLE` 문을 별도 파일(예: `db/migrations/`)로 작성합니다. 운영 단계에서는 Alembic 같은 마이그레이션 도구 도입을 검토합니다.

6.1 변경 기록 (migrations)

파일	내용	실행 대상
`migrations/2026-10-02_contact_devnotes.sql`	`inquiries.name` NULL 허용 + `privacy_agreed_at` 추가, `dev_notes` 생성	2026-10-02 이전에 01~06 을 실행한 DB (로컬 DB 실행 완료)